

STREAM NAVIGATOR — INSURANCE ENGINE AUDIT REPORT (Follow-Up)

StreamNavigator — Engine Audit
2026-09-20

A status check against ``docs/INSURANCE-AUDIT.md``, the original audit (2026-09-20): what it found, what shipped in response, what was verified and how, and what is still open. Every claim below is checked against the current ``main`` (``ba97039``) and, where marked, against production directly — not inferred from the original report's recommendations.

Executive Summary

The original audit found one central defect — ``insurance`` sold a renewal-vs-prior-policy comparison and a verdict on whether a premium increase is "typical," delivered by a single ungrounded model call with no deterministic engine, no rate data, and an optional (not required) prior policy that let the core comparison silently not run. Three Critical fixes were named as conditions on the product's \$79 price.

All three have shipped, and all three are verified working — two by direct live testing against production, one (extraction quality on a real document) still blocked on the same account-credit exhaustion the original audit disclosed. Every High-priority fix shipped too. Of the two Medium/Low items, one shipped (the hero's proof-of-mechanism example) and two were deliberately deferred as judgment calls, documented at the time rather than silently dropped.

Beyond the original recommendations, three things were added that the original report did not ask for but that materially reduce risk going forward: a cross-fixture regression test pinning the "no typical/market claim" rule so it can't quietly regress, a documented pricing note recording why \$79 stands, and a full offline verification harness (test documents with known answers, a diff tool, and a live-extraction script) so the one remaining unverifiable question — does the model read a real document correctly — is one command away the moment credits exist, rather than a rebuild.

Bottom line: the product now matches what the audit said it needed to be, as far as anything can be verified without spending money Anthropic has not had in the account for a week. The \$79 price stands on the terms the original audit set.

Fix-by-Fix Status

#	Original finding	Fix shipped	Evidence	Status
Critical 1	No deterministic engine; a single ungrounded model call did the comparison	<code>`api/_lib/insurance-extract.js`</code> (structured transcription) + <code>`api/_lib/insurance-audit.js`</code> (deterministic diff/categorization, no market-comparison evidence kind in its vocabulary)	Commit <code>`67f6f2a`</code> . 12 unit tests in <code>`tests/insurance-audit.test.js`</code> , all passing, against hand-computed fixtures	Closed

#	Original finding	Fix shipped	Evidence	Status
Critical 2	The page and prompt claimed a "typical" verdict nothing backs	"Typical" removed from `insurance.html`'s copy and from the task prompt; engine has no market/typical `EvidenceKind` by construction	Commit `67f6f2a`, `b4f3412`. A dedicated test (`tests/insurance-audit.test.js`) now scans every finding across every fixture and fails if any claims a premium is typical or matches the market	Closed, and now regression-guarded
Critical 3	Prior policy was optional; a customer could pay full price for a comparison that couldn't run	Two required upload zones on `insurance.html`; server-side gate in `api/navigator-intake.js` rejecting `attachmentCount < 2`	Commit `b534597`. Verified live on production: empty submission `!` 400, renewal-only `!` 400, both documents `!` 200 (curl against `streaknavigator.ai`, 2026-09-20)	Closed, verified live
High 4	No encoded justified-vs-worth-challenging rule set	Encoded directly in `insurance-audit.js`'s headline logic: coverage cut `!` worth-challenging, coverage/deductible improvement `!` likely-justified, no explanation `!` worth-challenging	Commit `67f6f2a`	Closed

#	Original finding	Fix shipped	Evidence	Status
High 5	No risk-protection guardrail against recommending coverage reduction	Coverage-gap findings (a dropped or reduced limit, a new exclusion) always rank first, regardless of premium direction; the write-up prompt explicitly forbids recommending coverage reduction to save money	Commit `67f6f2a`	Closed
High 6	No discipline distinguishing in-document arithmetic from an estimate	No savings estimate is ever produced — the engine states only the dollar/percent change between the two documents, explicitly labeled as arithmetic, never as a saving	Commit `67f6f2a`	Closed
Medium 7	No proof the mechanism works before a visitor pays	Hero now shows a labeled "Example finding" card — a real finding shape (dwelling limit cut, premium risen) the engine can actually produce	Commit `b4f3412`. Verified live in-browser, desktop and mobile	Closed
Medium 8	Visual language doesn't match `closing`'s seriousness	Deliberately not done — closing itself is the outlier from the shared design system used by ten other products; a full re-skin would fragment the site further. Judgment call, documented at the time	—	Deferred by design, not an oversight
Low 9	FAQ vague on which categories are actually well-supported	Not done	—	Open, low priority

What Was Verified, and How

Deterministic engine — offline, zero config
 intake-insurance.test.js` (4 tests) pin even

justified, within-norms, and the no-baseline insurance-audit-harness.js` runs the same as a `--compare` mode that diffs a captured diff, both a clean match (zero diffs) and a deliberate coverage limit and a missed discount, including of a finding that depended on it).

Intake gate — offline and live. Unit-test directly against production with three live intake`: an empty submission, a renewal-submission exactly as designed, with the deployed code.

Deployment. Every commit in this list comes treated as live, via the Vercel API.

Page and UI. Both upload zones, the validation confirmed live in-browser at `streamnavigation`.

End-to-end submission flow. A real test against production, flipped to `paid` in Suite, the extraction call fires correctly, the prompt returned `400: "Your credit balance is too low" account has returned on every attempt since products. It is account state, not a defect provider-outage.js`s classifier correctly leaves marking it failed.

What Remains Unverifiable

Three things, all downstream of the one being testing:

1. Whether `insurance-extract.js` reads a **real** renewal notice and prior policy correctly — coverage lines, deductibles, exclusions, premium, and correctly telling which document is which.
1. Whether the write-up model follows the new prompt in practice on a real run — categories carried through unchanged, no invented figures, the "never call it typical" rule actually holding under a live call rather than just in the prompt text.
1. The full render of a real insurance report on the status page and in the emailed PDF (architecturally proven on six other products using the same shared renderer, but unproven with this product's specific content).

This is now the smallest it can get without spending money. `scripts/make-test-documents.js` generates two PDFs — a renewal notice and a prior-policy declarations page — printing known, planted

figures (the same ones ``tests/fixtures/insurance-fixtures.js``'s ``homeCoverageCutBehindTheRise()`` already uses as its hand-authored ground truth, so there is nothing new to keep in sync). ``scripts/live-test-insurance.js`` calls the extraction function directly against those two PDFs — one model call, no Supabase, no production involvement — the moment ``ANTHROPIC_API_KEY`` has a balance. ``scripts/insurance-audit-harness.js --compare`` then grades the result against ground truth automatically. Three commands closes item 1 above; the same harness run against a captured real production extraction would close it with an actual customer document instead of a synthetic one.

Pricing

Current price: \$79/renewal, unchanged, now documented in ``prices.config.json`` with a ``_price`` note recording that all three Critical conditions the original audit set have shipped — matching the standard already applied to Landlord (\$149) and Government Money (\$39): the price was held, not cut, once its conditions were met.

No change recommended. The one thing that would still justify revisiting it — real-document extraction quality — is exactly the piece blocked on credits, not on anything this report can settle by further analysis.

Updated Customer-Value Verdict

If I were a rational customer paying \$79, would I reasonably expect this product to save me more money than it costs while helping me avoid a materially worse insurance decision?

More defensible than at the original audit, not yet provable end to end. The product's mechanism — a genuine diff between two documents, categorized by rules that never recommend cutting coverage to save money, that never claims a premium is "typical" — is now real and independently tested, not asserted. What still cannot be shown is whether that mechanism reads a real customer's actual paperwork with enough fidelity for the categorization to be trustworthy in practice. That is a fair, disclosed gap, closeable in three commands once the account has credits, and it is the only thing standing between "the audit's conditions are met" and "the product is proven end to end."